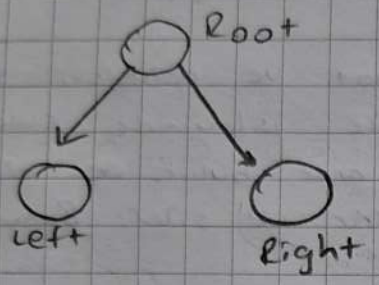
Degenerate = Déjenere = Bozulmuş

24.04.2026 (10. Hafta)

Ağaç (Tree) yapısı → farklı versiyonlar var.
BST → Her düğümün en fazla 2 çocuğu var ve sol düğüm kökten küçük, sağ ise büyüktür.

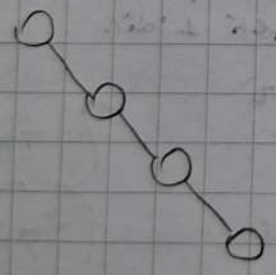


Left < Root < Right

BST'de aslında tek kuralımız bu.

→ insert → $\underbrace{O(1)}_{\text{best case}}$, $\underbrace{O(\log n)}_{\text{dengeli (balaced) sağlamış}}$ → Bunun $O(n)$ olduğu yani worst case durumunun ihtimali de var.

* Giriş sırasına göre BST liste gibi davranabiliyor ve karmaşıklık $O(n)$ oluyor. Bu da worst case'dir. Buna "Degenerate Tree" deniyor ve liste gibi davranıyor. Peki bunun ihtimali nedir?
Örneğin; 4-7-9-20-40 girişlerinin worst case olabilmesi için ihtimal nedir?



worst-case'de bu şekilde bir ağaç olur.

Sonuçta worst case için tek bir sıralamada olmalı (1 durum) Bu sayılar 5! kadar farklı şekilde sıralanabilir. O halde bu ihtimal n!'de 1'dir.
$\left(\frac{1}{n!}\right)$

↳ 100 düğüm olsa 100!'de 1 ihtimal $O(n)$'e gidecek. Bu risk alınabilir!!

↳ Yani BST'nin worst-case ($O(n)$) ihtimali $\frac{1}{n!}$'dir.

* BST worst case, ağacın tamamen dengesiz hale gelerek tek yönlü doğrusal bir yapıya (dejenere ağaç) dönüşmesidir. Yani worst case durumu dengesiz bir ağaç.

* Bu konular BST animasyonlu siteden (cs.usfca.edu) işledik
* Data Structures and algorithms in Java kitabına bak.
* MyUSF sitesini incele → BST animasyonu.

* Best case durumu da dengeli ağaçtır. Burada yine veri geliş sırası önemli.
* Aynı verileri farklı sırada girerek dengeli bir ağaç oluşturabiliriz.

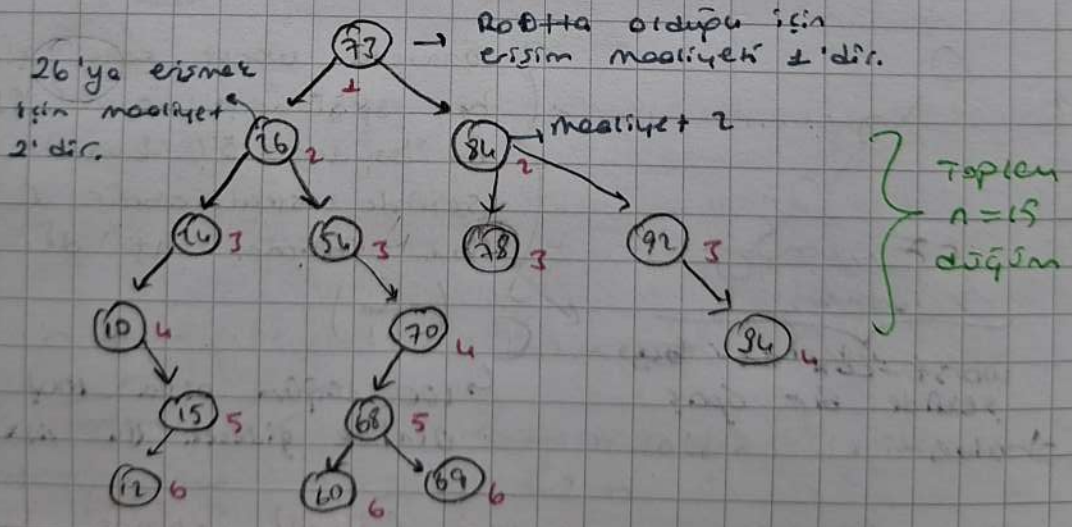
* delete / remove :

delete (40) dediğimizde 40 root ve ağaçtaki en sağda olan düğüm 35 ise bu delete işleminde swap(40,35) yapılır. En büyük eleman olan 35'i arar bulur.
Daha sonra 35 ve 40'ı yer değiştirir ve siler. Bunun karmaşıklığı için:

find (search → O(log n)
swap → O(1)
} Bu rasyonel toplamı O(log n) olur.

O halde delete için karmaşıklık O(log n) 'dir.
↳ Bunun worst case ihtimali O(n) karmaşıklık, $\frac{1}{n!}$ ihtimaldir.

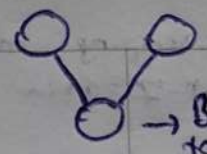
Average-case (BST için) → Bunu kitaptan konuştuk.
* Ağacın tüm npl değerlerini topladığımızda bunlar her bir düğüme olan erişim maliyetini bize verir. Aynı zamanda ortalama bize beklentiyi (ortalama beklenti) verir. N düğümlü bir ağaçta beklentiyi konuşucaz.



Toplam npl → 1+2+2+3+3+3+3+3.4+2.5+3.6 = 57

$\underbrace{57}_{\text{Toplam npl}} / \underbrace{15}_{n} \cong 4$ → ortalama 4 adımda düğümlere erişebiliyoruz.

Toplamı düğüm sayısına böldük.



→ Bu düğümü toplam npl'de 2 kez sayarız.

* Root'a gelmenin maliyeti → 1

* Derste yaptığımız uygulamanın ürettiği farklı ağacın ortalama beklentisini hesapladık. Düğüm sayısını 14 verdik.

↳ $1 + 2 + 2 \cdot 3 + 3 \cdot 4 + 2 \cdot 5 + 2 \cdot 6 + 2 \cdot 7 = 57$ → Toplam npl değeri

↳ $\underbrace{57}_{\text{Toplam npl}} / \underbrace{14}_{\text{Toplam düğüm}} \cong 4$ → ortalama derinlik 4 oldu.

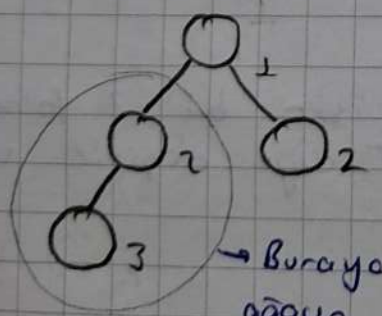
↳ Burada bir BST'deki beklentileri bulduk.

* 15 düğüm için 4 değeri 15'in $\log n$'idir. Denemeler yaparak da bu değerleri bularak 4'ü elde ettik. Bunu gördük. Denemelerde balanced, unbalanced durumunda 4 geldiğini gördük (15 node için). Kitapta sayfa 320'de bunun matematiği var:

$$T(N) = \frac{2}{N}\left[\underbrace{\sum_{j=0}^{N-1} T(j)}_{\text{her bir durumunu topluyor.}}\right] + \underbrace{cN}_{\text{quick sort mantığında } N \text{ sayıya bakılmalı. Bu maliyet oradan geliyor.}}$$

* BST'nin average case complexity'si ile quick sortun average case'inin matematiği benzer
↳ BST average case ~ quick sort average

Bir ağaç yapısında toplam yol / path sayısı nedir?

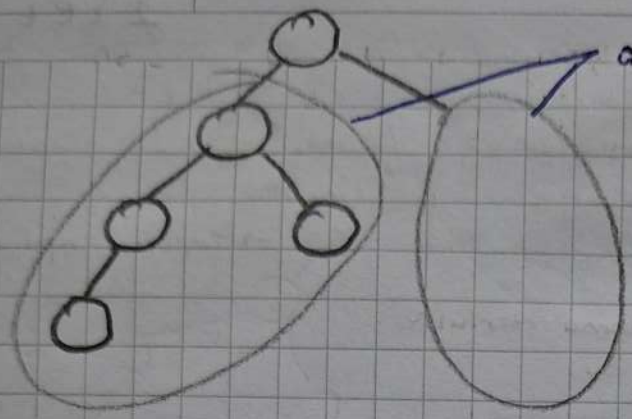


→ Root npl 1 olsun. Burada toplam npl değeri kaçtır?
Toplam npl → $1 + 2 + 2 + 3 = 8$ oldu.

→ Buraya alt ağaç dersek bu alt ağacın toplam npl değeri 5'tir.

* N tane düğümün oluşturduğu bir ağaç yapısını 1 root'a bağlarsak $D(N)$ değeri $+(N-1)$ kadar artar. Burada $D(N)$, alt ağaçtaki toplam npl değeridir.

Telescoping mantığında uygun forma getirip N-1, N-2, için yazmak vardır.



0	N-1
1	N-2
2	N-3
⋮	⋮
N-1	0

} ikisi birbirinin simetriğidir.

O yüzden buradaki sayıyı 0'dan N-1'e kadar götürüyoruz. Yani aşağıdaki formülü açıkladık:

$$D(N) = \frac{2}{N}\left[\underbrace{\sum_{j=0}^{N-1} D(j)}\right] + \underline{N-1}$$

Bir root'u bağlıyormuş gibi düşünüp bunu ekliyoruz. Bunun için hepsine 1 ekledik.

Bu şekilde her bir D(j), D(N) için farklı farklı değerler elde ediyor.

$$T(N) = \frac{2}{N}\left[\sum_{j=0}^{N-1} T(j)\right] + c.N$$

BST ve Quick sortun average'i için bu 2 formül benzerdir. Quick sort için D'ler T diye değişir ve N-1 yerine c.N yazılır.

Average-Case Analysis

- Toplam bekleme sayısı için telescoping yapıyoruz.

$$N./\ T(N) = \frac{2}{N}\left[\sum_{j=0}^{N-1} T(j)\right] + c.N \quad (1)$$

→ $\frac{1}{N}$'i yok etmek için

(1) → başlangıçta N ile çarptık.

$$N\,T(N) = 2\left[\sum_{j=0}^{N-1} T(j)\right] + cN^2$$ elde ettik

N yerine N-1 yazalım

$$(N-1)\,T(N-1) = 2\left[\sum_{j=0}^{N-2} T(j)\right] + c(N-1)^2 \qquad (2)$$

(1)'den (2)'yi çıkarınca sigmalar gider ve sade bir hal kalır.

$$N\,T(N) - (N-1)\,T(N-1) = 2\,T(N-1) + 2cN - c$$

Buradan $N\,T(N) \cong$ 7.18'e ulaştık.

$N\,T(N)$'yi telescoping yapmak için $\frac{1}{N.(N+1)}$ ile çarp

$$\frac{T(N)}{N+1} = \frac{T(N-1)}{N} + \frac{2c}{N+1} \qquad (3)$$

Bundan sonra aynı ifadeyi $N-1, N-2, N-3, \ldots, 2$ için yazıp hepsini taraf tarafa toplarız.

$$\frac{T(N)}{N+1} = \underbrace{\frac{T(1)}{2}}_{\text{sbt.}} + \underbrace{2c\sum_{i=3}^{N+1}\frac{1}{i}}_{O(\log n)}$$

$2c\sum_{i=3}^{N+1}\frac{1}{i}$ değeri $\log(N+1) + \gamma - \frac{3}{2}$ 'dir. Yani bu değer $O(\log n)$'e gidiyor.

$$\frac{T(N)}{N+1} = O(\log N)$$

$T(N) = O(N\log N)$ → Bu az önce hesapladığımız npl değerini veriyor.

$\frac{T(N)}{N} = \frac{O(N\log N)}{N}$ → Bu da toplam bekleti sayısını veriyor.

! Depth = NPL

* Bu şekilde hem Quick hem BST için average case'i oradan çıkarıyor.
* Quick sort worst case'de $O(n^2)$'ye gidebilir ama average'da $n\log n$'e gider.
* BST beklenti için $\log n$'dir.
* Tekrar sayısını arttırdıkça gerçek ortalamaya yaklaşmaya central limit teorem deriz. Biz bu şekilde de karmaşıklıkları ispatladık.
* 30 node'da $2^5 = 32$ olduğundan dolayı 5'e yaklaşık bir değer bekleriz. 60 olsa 2^6 olacaktı. Yani ortalamayı 6 beklerdik. (log'u burada 2 tabanda alıyoruz)
* Heap sortta ekstra algoritma gerek. Quick sortta böyle bir şey yok.

4 Bu derste ikili arama ağacı kodu, animasyonu, kitaptan matematiğini konuştuk.